

Domain Overview

1. Wallet Domain Overview

- What is **wallet** is system?
A wallet in this system represents a **logical financial ledger** that tracks monetary value using immutable credit and debit records.
- What problem does it solve?
The primary problem this system solves is ensuring correctness, auditability, and reliability of financial state changes.
Traditional CRUD-based designs make it difficult to trace how a balance was reached, handle concurrent updates safely, or recover state after failures.
By modelling the wallet as an **event-driven ledger**, the system guarantees that every financial change is recorded, traceable, and reproducible.
- Why this domain was chosen?
The wallet domain was chosen because it has strict **business invariants**, such as preventing negative balances and duplicate debits, which makes it ideal for demonstrating event sourcing and **domain-driven design principles**.

2. Definition of Wallet

A wallet in this system is a logical financial entity identified by a **unique wallet identifier** that represents ownership of a monetary value over time. It conceptually contains the wallet's identity, a monetary balance derived from **past financial actions**, and a complete history of events that explain how that balance was reached. The wallet **does not store balance as a mutable field**, instead its current value is the result of applying all recorded financial events in order. This wallet is strictly a **backend domain concept and does not represent any user interface**, screen, or visual account shown to an end user. It is also **not a payment gateway or external payment processor**, but rather the internal ledger system responsible for maintaining financial correctness, traceability, and auditability of monetary state changes.

3. Core Capabilities

The wallet domain provides a minimal but strictly **controlled set of business** capabilities focused on maintaining financial correctness throughout the wallet lifecycle. The system allows the creation of a wallet as an initial step, after which the wallet may transition through defined states such as activation, suspension, or closure before financial activity is permitted. Once active, a wallet can be credited with funds, representing an increase in monetary value, or debited, representing a reduction in value, provided all business invariants are satisfied. In cases where a previously recorded financial action must be corrected, the system **supports reversal operations** that compensate for earlier credit or debit events **without altering historical data**. The system also supports **querying** wallet-

related information such as the current balance and historical transaction records, where the **balance is always derived from recorded financial events rather than stored directly**. These capabilities are intentionally limited to ensure that every supported action can be validated, audited, and safely replayed without violating domain rules.

4. Wallet Invariants

Invariants = **rules** that must never be violated, even under **failure**.

- Wallet must exist before any transaction
- A wallet must be in an active state before credit or debit operations are allowed.
- Wallet balance must never be negative
- A debit operation must not execute more than once
- Transaction order must be preserved
- Once a wallet is suspended or closed, no new credit or debit operations may be applied.
- Reversal operations must only compensate for previously recorded credit or debit actions and must not alter or delete historical events.

5. Domain Terminology

Wallet: A wallet is a logical container that represents ownership of monetary value and maintains a record of all financial changes associated with it.

Balance: Balance is the current amount of money in a wallet, calculated from all past credit and debit actions rather than stored as a direct value.

Credit: Credit is an action that increases the balance of a wallet by adding monetary value to it.

Debit: Debit is an action that decreases the balance of a wallet by removing monetary value from it, provided sufficient balance is available.

Transaction: A transaction represents a single financial change applied to a wallet, such as a credit or a debit reversal, along with its relevant details.

Event: An event is a factual record that indicates something has already happened in the system, such as a wallet being credited or debited or activated etc.

Command: A command is a request or intention to perform an action on a wallet, such as requesting a credit or debit, which may either succeed or fail based on domain rules.

6. Domain Guarantees

- All financial actions are **auditable**
- Historical data is **immutable**
- Balance can always be **reconstructed**
- Data is always **consistent**
- **High Performance** and **Scalable** Systems

7. Domain Non-Guarantees

- Immediate consistency for reads
- Cross-wallet atomic transfers
- Real-time settlement with banks

8. Why Event Sourcing

- Why **Not CRUD**, Why **Event Driven Architecture**?

Financial systems require a high level of correctness, traceability, and trust, which makes traditional CRUD-based designs risky. In a CRUD approach, financial state such as balance is continuously overwritten, making it difficult to understand how a value was reached, detect incorrect updates, or **recover accurately after failures**. Once data is modified or lost, reconstructing the true financial history becomes complex and error-prone.

- Why immutability matters?

Immutability is critical in the wallet domain because **financial actions should represent facts that cannot be changed after they occur**. By recording immutable events, the system ensures that past actions remain reliable and tamper-resistant. Mistakes or corrections are handled by recording new compensating actions rather than altering historical data, preserving the integrity of the ledger.